

```

1  import flet as ft
2  import paho.mqtt.client as mqtt
3  import random
4  import datetime
5  import threading
6  import time
7  import json
8  import os
9  import tempfile # Garante um diretório com permissão de escrita em qualquer SO
10
11  # --- CONFIGURAÇÕES MQTT ---
12  MQTT_SERVER = "emqx.ifspb-czemnumeros.com.br"
13  MQTT_PORT = 1883
14  MQTT_USER = "esp12_01"
15  MQTT_PASS = "esp12_01"
16
17  # Dicionário global para controlar o estado das automações
18  automacoes = {
19      "casa/rele1": {"timer_minutos": 0, "hora_agenda": "18:00", "dias_agenda": set(),
20      "timer_objeto": None},
21      "casa/rele2": {"timer_minutos": 0, "hora_agenda": "18:00", "dias_agenda": set(),
22      "timer_objeto": None}
23  }
24
25  # --- DEFINIÇÃO DO CAMINHO COMPATÍVEL MULTIPLATAFORMA ---
26  # tempfile.gettempdir() aponta para uma pasta com permissões totais tanto no PC
27  # quanto no Android/iOS
28  CONFIG_FILE = os.path.join(tempfile.gettempdir(), "config_automacoes_jardim.json")
29  print(f"DEBUG: Caminho seguro utilizado = {CONFIG_FILE}")
30
31  def main(page: ft.Page):
32      page.title = "Meu Jardim - Automação"
33      page.theme_mode = ft.ThemeMode.DARK
34      page.window_width = 400
35      page.window_height = 700
36      page.horizontal_alignment = "center"
37
38      # --- BARRA SUPERIOR ---
39      page.appbar = ft.AppBar(
40          title=ft.Text("🏠 AUTOMAÇÃO", size=22, weight="bold"),
41          center_title=True,
42          bgcolor="surface",
43      )
44
45      # --- STATUS DO BROKER ---
46      status_led = ft.Container(width=12, height=12, bgcolor="red", border_radius=6)
47      status_text = ft.Text("Desconectado", color="red", weight="bold")
48
49      # --- CLIENTE MQTT ---
50      client_id = f'flet-mqtt-{random.randint(0, 1000)}'
51      mqttc = mqtt.Client(client_id=client_id)
52      mqttc.username_pw_set(MQTT_USER, MQTT_PASS)
53
54      def on_connect(client, userdata, flags, rc):
55          if rc == 0:
56              status_led.bgcolor = "green"
57              status_text.value = "Online"
58              status_text.color = "green"
59          else:
60              status_led.bgcolor = "red"
61              status_text.value = f"Erro {rc}"
62              status_text.color = "red"
63          page.update()
64
65      mqttc.on_connect = on_connect
66
67      # --- FUNÇÃO ATUADORA (MUDAR RELE) ---
68      def mudar_rele(e):
69          topic = e.control.data
70          msg = "ON" if e.control.value else "OFF"
71
72          if msg == "OFF" and automacoes[topic]["timer_objeto"] is not None:
73              try:

```

```

71         automacoes[topic]["timer_objeto"].cancel()
72         automacoes[topic]["timer_objeto"] = None
73     except Exception:
74         pass
75
76     try:
77         mqttc.publish(topic, msg)
78     except Exception as err:
79         print(f"Erro MQTT: {err}")
80
81 # --- PERSISTÊNCIA LOCAL ---
82 def salvar_dados_no_disco():
83     try:
84         dados_para_salvar = {
85             "casa/rele1": {
86                 "timer_minutos": automacoes["casa/rele1"]["timer_minutos"],
87                 "hora_agenda": automacoes["casa/rele1"]["hora_agenda"],
88                 "dias_agenda": list(automacoes["casa/rele1"]["dias_agenda"])
89             },
90             "casa/rele2": {
91                 "timer_minutos": automacoes["casa/rele2"]["timer_minutos"],
92                 "hora_agenda": automacoes["casa/rele2"]["hora_agenda"],
93                 "dias_agenda": list(automacoes["casa/rele2"]["dias_agenda"])
94             }
95         }
96         with open(CONFIG_FILE, "w") as f:
97             json.dump(dados_para_salvar, f)
98             print("DEBUG: Dados guardados em segurança!")
99     except Exception as e:
100         print(f"Erro ao salvar dados: {e}")
101
102 # --- LÓGICA DO TEMPORIZADOR (TIMER) ---
103 def desligar_por_timeout(pino_rele, switch_componente):
104     try:
105         mqttc.publish(pino_rele, "OFF")
106         switch_componente.value = False
107         automacoes[pino_rele]["timer_objeto"] = None
108         automacoes[pino_rele]["timer_minutos"] = 0
109
110         page.snack_bar = ft.SnackBar(ft.Text(f"🔌 {pino_rele} desligado! Tempo terminou."))
111         page.snack_bar.open = True
112         page.update()
113     except Exception as e:
114         print(f"Erro no timeout do timer: {e}")
115
116 def acionar_temporizador(pino_rele, txt_timer, switch_componente):
117     try:
118         minutos = int(txt_timer.value)
119     except ValueError:
120         page.snack_bar = ft.SnackBar(ft.Text("Insira um número válido de minutos!"))
121         page.snack_bar.open = True
122         page.update()
123         return
124
125     if minutos <= 0:
126         page.snack_bar = ft.SnackBar(ft.Text("O tempo deve ser maior que zero!"))
127         page.snack_bar.open = True
128         page.update()
129         return
130
131     if automacoes[pino_rele]["timer_objeto"] is not None:
132         automacoes[pino_rele]["timer_objeto"].cancel()
133
134     mqttc.publish(pino_rele, "ON")
135     switch_componente.value = True
136
137     page.snack_bar = ft.SnackBar(ft.Text(f"🕒 Temporizador iniciado por {minutos} minuto(s)"))
138     page.snack_bar.open = True
139     page.update()
140

```

```

141     segundos = minutos * 60
142     t = threading.Timer(segundos, desligar_por_timeout, args=[pino_rele,
143         switch_componente])
144     automacoes[pino_rele]["timer_objeto"] = t
145     automacoes[pino_rele]["timer_minutos"] = minutos
146     t.start()
147
148     salvar_dados_no_disco()
149
150     # --- INPUTS VISUAIS ---
151     txt_timer_d6 = ft.TextField(label="Minutos", value="30", width=90,
152         text_align="center", dense=True)
153     txt_hora_d6 = ft.TextField(label="HH:MM", value="18:00", width=90,
154         text_align="center", dense=True)
155     buttons_d6 = {}
156
157     txt_timer_d7 = ft.TextField(label="Minutos", value="30", width=90,
158         text_align="center", dense=True)
159     txt_hora_d7 = ft.TextField(label="HH:MM", value="18:00", width=90,
160         text_align="center", dense=True)
161     buttons_d7 = {}
162
163     switch_luz = ft.Switch(value=False, data="casa/rele1", on_change=mudar_rele)
164     switch_tomada = ft.Switch(value=False, data="casa/rele2", on_change=mudar_rele)
165
166     # --- CARREGA DADOS DO ARQUIVO ANTES DE CRIAR INTERFACE ---
167     def carregar_dados_local():
168         try:
169             if os.path.exists(CONFIG_FILE):
170                 with open(CONFIG_FILE, "r") as f:
171                     dados_dict = json.load(f)
172                     print(f"DEBUG: Dados carregados: {dados_dict}")
173
174                     for pino in ["casa/rele1", "casa/rele2"]:
175                         if pino in dados_dict:
176                             automacoes[pino]["hora_agenda"] =
177                                 dados_dict[pino].get("hora_agenda", "18:00")
178                             automacoes[pino]["timer_minutos"] =
179                                 dados_dict[pino].get("timer_minutos", 0)
180                             dias_carregados = dados_dict[pino].get("dias_agenda", [])
181                             automacoes[pino]["dias_agenda"] = set(dias_carregados)
182         except Exception as e:
183             print(f"Erro ao carregar dados: {e}")
184
185     carregar_dados_local()
186
187     # --- CONSTRUTOR DE BLOCOS ---
188     def criar_bloco_automacao(pino_rele, txt_timer, txt_hora, buttons_dict,
189         switch_componente):
190         def salvar_agenda(e):
191             automacoes[pino_rele]["hora_agenda"] = txt_hora.value
192             salvar_dados_no_disco()
193             page.snack_bar = ft.SnackBar(ft.Text(f"Agenda de {pino_rele} salva para
194                 as {txt_hora.value}!"))
195             page.snack_bar.open = True
196             page.update()
197
198         def alternar_dia_btn(e):
199             container_alvo = e.control.content
200             dia = container_alvo.data
201
202             if dia in automacoes[pino_rele]["dias_agenda"]:
203                 automacoes[pino_rele]["dias_agenda"].discard(dia)
204                 container_alvo.bgcolor = "transparent"
205                 container_alvo.border = ft.Border(
206                     ft.BorderSide(1, "white30"), ft.BorderSide(1, "white30"),
207                     ft.BorderSide(1, "white30"), ft.BorderSide(1, "white30")
208                 )
209             else:
210                 automacoes[pino_rele]["dias_agenda"].add(dia)
211                 container_alvo.bgcolor = "blue"
212                 container_alvo.border = ft.Border(
213                     ft.BorderSide(1, "blue"), ft.BorderSide(1, "blue"),

```

```

205         ft.BorderSide(1, "blue"), ft.BorderSide(1, "blue")
206     )
207
208     salvar_dados_no_disco()
209     page.update()
210
211     dias_semana = ["Seg", "Ter", "Qua", "Qui", "Sex", "Sáb", "Dom"]
212     lista_botoes = []
213
214     for d in dias_semana:
215         esta_selecionado = d in automacoes[pino_rele]["dias_agenda"]
216         bgcolor = "blue" if esta_selecionado else "transparent"
217         cor_borda = "blue" if esta_selecionado else "white30"
218
219         container_chip = ft.Container(
220             content=ft.Text(d, size=11, weight="bold", color="white"),
221             alignment="center",
222             padding=8,
223             width=45,
224             height=32,
225             border_radius=6,
226             border=ft.Border(
227                 ft.BorderSide(1, cor_borda), ft.BorderSide(1, cor_borda),
228                 ft.BorderSide(1, cor_borda), ft.BorderSide(1, cor_borda)
229             ),
230             bgcolor=bgcolor,
231             data=d
232         )
233
234         detector_clique = ft.GestureDetector(
235             content=container_chip,
236             on_tap=alternar_dia_btn
237         )
238
239         buttons_dict[d] = container_chip
240         lista_botoes.append(detector_clique)
241
242     txt_hora.value = automacoes[pino_rele]["hora_agenda"]
243     if automacoes[pino_rele]["timer_minutos"] > 0:
244         txt_timer.value = str(automacoes[pino_rele]["timer_minutos"])
245
246     return ft.ExpansionTile(
247         title=ft.Text("Configurar Temporizador e Agenda", size=13,
248             color="blue200"),
249         maintain_state=True,
250         controls=[
251             ft.Container(
252                 padding=10,
253                 bgcolor="black26",
254                 border_radius=8,
255                 content=ft.Column([
256                     ft.Row([
257                         ft.Icon(ft.Icons.TIMER, color="amber"),
258                         ft.Text("Desligar em:", weight="w500"),
259                         txt_timer,
260                         ft.IconButton(
261                             icon=ft.Icons.PLAY_ARROW_ROUNDED,
262                             icon_color="green",
263                             on_click=lambda e: acionar_temporizador(pino_rele,
264                                 txt_timer, switch_componente)
265                         )
266                     ], alignment="spaceBetween"),
267
268                     ft.Divider(height=10, color="white10"),
269
270                     ft.Row([
271                         ft.Icon(ft.Icons.SCHEDULE, color="blue"),
272                         ft.Text("Ligar às:", weight="w500"),
273                         txt_hora,
274                         ft.IconButton(icon=ft.Icons.SAVE, icon_color="blue",
275                             on_click=salvar_agenda)
276                     ], alignment="spaceBetween"),

```

```

275         ft.Text("Dias da semana:", size=12, color="white60"),
276         ft.Container(
277             height=100,
278             content=ft.Row(lista_botoes, wrap=True,
279                             alignment="center", spacing=5)
280         ),
281     ], spacing=10)
282 )
283 )
284
285 # --- MONTAGEM DOS CARDS ---
286 card_luz = ft.Card(
287     content=ft.Container(
288         padding=12,
289         content=ft.Column([
290             ft.Row([
291                 ft.Row([ft.Icon(ft.Icons.LIGHTBULB, color="amber", size=28),
292                         ft.Text("Luz (D6)", size=18, weight="w500")]),
293                 switch_luz
294             ], alignment="spaceBetween"),
295             criar_bloco_automacao("casa/rele1", txt_timer_d6, txt_hora_d6,
296                                   buttons_d6, switch_luz)
297         ])
298     )
299 )
300
301 card_tomada = ft.Card(
302     content=ft.Container(
303         padding=12,
304         content=ft.Column([
305             ft.Row([
306                 ft.Row([ft.Icon(ft.Icons.POWER, color="blueGrey200", size=28),
307                         ft.Text("Tomada (D7)", size=18, weight="w500")]),
308                 switch_tomada
309             ], alignment="spaceBetween"),
310             criar_bloco_automacao("casa/rele2", txt_timer_d7, txt_hora_d7,
311                                   buttons_d7, switch_tomada)
312         ])
313     )
314 )
315
316 # --- RENDERIZAÇÃO DA INTERFACE ---
317 page.add(
318     ft.Container(height=15),
319     ft.Row([status_led, status_text], alignment="center"),
320     ft.Divider(),
321     card_luz,
322     card_tomada
323 )
324
325 page.update()
326
327 # --- PROCESSAMENTO ASSÍNCRONO EM BACKGROUND THREAD ---
328 def conectar_mqtt_e_agendar():
329     try:
330         mqttc.connect(MQTT_SERVER, MQTT_PORT, 60)
331         mqttc.loop_start()
332     except Exception:
333         pass
334
335 def loop_verificacao_horario():
336     while True:
337         agora = datetime.datetime.now()
338         dias_map = ["Seg", "Ter", "Qua", "Qui", "Sex", "Sáb", "Dom"]
339         dia_atual_str = dias_map[agora.weekday()]
340         hora_atual_str = agora.strftime("%H:%M")
341
342         for pino, dados in automacoes.items():
343             if dados["hora_agenda"] == hora_atual_str and dia_atual_str in
344                 dados["dias_agenda"]:
345                 mqttc.publish(pino, "ON")
346                 if pino == "casa/rele1":
347                     switch_luz.value = True

```

```
342         if pino == "casa/rele2":
343             switch_tomada.value = True
344             page.update()
345
346         time.sleep(60)
347
348     threading.Thread(target=conectar_mqtt_e_agendar, daemon=True).start()
349     threading.Thread(target=loop_verificacao_horario, daemon=True).start()
350
351 ft.app(main)
```